
The Job Sequencing and Tool Switching Problem

Part VIII: Branch-and-Benders Decomposition (BBC)

A master sequence problem with a polynomial KTNS subproblem

Scope and status. This note develops the Branch-and-Benders-cut (BBC) exact method for the uniform SSP. The SSP splits cleanly into a *sequencing* decision and a *tooling* decision: fix the job sequence and the optimal tooling cost is computed in polynomial time by KTNS [Tang and Denardo(1988)]. BBC puts the sequence in a *master* ATSP (with a surrogate θ for the tool cost) and the tooling in a *subproblem* solved by KTNS, linked by Benders optimality cuts. The decisive structural feature — and the reason BBC is attractive here — is that **the subproblem is polynomial**, in contrast to the Dantzig–Wolfe/column-generation route of Part II whose pricing subproblem is itself an NP-hard SSP. The implementation (`src/BBC/branch_and_benders_cut_cplex.py`) is exact and supports *two* cut families: by default, classical **LP-duality Benders cuts** from the dual subproblem (DSP) of an exact tooling LP (section 3); optionally (`use_combinatorial_cuts`), the cheaper but weaker KTNS-based combinatorial cut (proposition 3.1). Because the DSP is an LP whose feasible region does not depend on x , valid cuts exist at *fractional* master points too; a relaxation-callback implementation exists but is untested — the open questions are empirical (section 6).

Contents

The Decomposition

Notation. Jobs $J = \{1, \dots, n\}$, tools T , capacity b , required sets $T_j \subseteq T$. A dummy job 0 with $T_0 = \emptyset$ acts as a free start/end depot, so the schedule is an open path over the real jobs (no tool cost is charged for loading the first configuration). For a job sequence σ , $\text{KTNS}(\sigma)$ denotes the minimum number of tool switches over σ , computed by the Keep-Tool-Needed-Soonest policy.

The split. A feasible SSP solution is a pair (sequence σ , tooling). For a *fixed* σ the optimal tooling is decided independently and exactly:

Theorem 1.1 (KTNS solves the subproblem exactly and in polynomial time [Tang and Denardo(1988)]). *For any fixed job sequence σ , the minimum number of tool switches equals $\text{KTNS}(\sigma)$, computable in $O(n|T|)$ time.*

Hence

$$Z_{\text{SSP}}^* = \min_{\sigma} \text{KTNS}(\sigma),$$

the minimisation being over the $n!$ job sequences (a permutation problem). BBC optimises this by branch-and-cut on a master that chooses σ , deferring the evaluation $\text{KTNS}(\sigma)$ to a callback.

Remark 1.2 (Why this split is the right one for the SSP). Two decompositions compete (cf. Parts II and IV). *Dantzig–Wolfe / column generation* (Part II) generates configurations as columns; its pricing subproblem is a set-union knapsack and, for whole sequences, an NP-hard SSP. *Benders* fixes the complicating variables (the sequence) and leaves a subproblem that here is *polynomial* (KTNS). Benders therefore guarantees that exact cuts can always be produced cheaply — the property the CG route lacks. The price is that the master’s lower bound is only as strong as the cuts accumulated so far.

The Master Problem

The master is an asymmetric TSP over $J \cup \{0\}$ with arc variables $x_{ij} \in \{0, 1\}$ ($x_{ij} = 1$ iff job j immediately follows job i) and a single continuous surrogate $\theta \geq 0$ standing for the tool-switching cost of the chosen sequence.

BBC master (relaxed; cuts added dynamically)

$$\min \theta \tag{1}$$

$$\text{s.t. } \sum_{j \neq i} x_{ij} = 1, \quad \sum_{i \neq j} x_{ij} = 1 \quad \forall i, j \in J \cup \{0\} \tag{2}$$

$$\begin{array}{ll} \text{(subtour elimination on } x) & \text{(lazy SEC or MTZ, remark 2.1)} \\ & \tag{3} \end{array}$$

$$\theta \geq \sum_{i \neq j} w_{ij} x_{ij}, \quad w_{ij} = \max(0, |T_i \cup T_j| - b) \tag{4}$$

$$\theta \geq \text{(Benders optimality cuts, section 3)} \tag{5}$$

$$x_{ij} \in \{0, 1\}, \quad \theta \geq 0.$$

Constraints (2)–(3) make x a Hamiltonian cycle through the depot, i.e. an open job sequence $\sigma(x)$. Inequality (4) is the *pairwise lower bound*: each executed transition $i \rightarrow j$ forces at least $w_{ij} = \max(0, |T_i \cup T_j| - b)$ switches, because the two jobs’ tools cannot both fit unless $|T_i \cup T_j| \leq$

b [Tang and Denardo(1988), Laporte et al.(2004)Laporte, Salazar-González, and Semet]. It is valid *a priori* and seeds the master with a non-trivial bound before any Benders cut is generated.

Remark 2.1 (Subtour elimination in the master). Any TSP subtour scheme works for (3): lazy DFJ cuts (separated on integer incumbents) or the polynomial MTZ constraints of Part IV (04_mtz_formulation.tex). Note the roles differ from Part IV: there MTZ replaced *configuration*-graph subtour constraints in a non-compact model; here it would only order the n jobs, where it is unproblematic and exact.

Benders Optimality Cuts

Although KTNS itself is a combinatorial algorithm, the tooling decision admits an exact *linear program*: for nodes $J \cup \{0\}$, variables $y_{jt} \geq 0$ (tool t in the magazine at job j) and $z_{jt} \geq 0$ (tool t inserted at j), with

$$\begin{aligned} \min \sum_{j \in J} \sum_{t \in T_j} z_{jt} \quad \text{s.t.} \quad & y_{jt} - y_{it} - z_{jt} \leq 1 - x_{ij} & \forall i \neq j \in J, t \in T, \\ & y_{jt} - z_{jt} \leq 1 - x_{0j} & \forall j \in J, t \in T \quad (y_{0t} \equiv 0), \\ & \sum_t y_{jt} \leq b; \quad y_{jt} = 1 \quad (t \in T_j); \quad z_{jt} = 0 \quad (t \notin T_j). \end{aligned}$$

The depot rows with $y_{0t} \equiv 0$ start the magazine *empty* and break the wrap-around (no row for $j \rightarrow 0$), so for an integer depot cycle $\bar{x}$ encoding σ this LP equals the optimal number of insertions from an empty magazine — verified to coincide with `compute_ktns( $\sigma$ )`, with *integral* LP optima, on 60/60 random pairs (`_verification/verify_bbc2.py`); the theoretical basis is the interval-matrix/TU integrality of the fixed-sequence tooling LP [Crama et al.(1994)Crama, Kolen, Oerlemans, and Spielmann; Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé] (TODO: pin the exact theorem variant for these persistence constraints). Its dual is the **DSP** of the implementation (variables $\lambda_{ijt} \geq 0$, $\mu_j \geq 0$, ν_{jt} free, η_{jt} free), with objective

$$\max \sum_{i \neq j, t} (\bar{x}_{ij} - 1) \lambda_{ijt} - b \sum_j \mu_j + \sum_j \sum_{t \in T_j} \nu_{jt}.$$

Since $\bar{x}$ enters only the *objective*, the dual feasible region is independent of x ; weak duality therefore gives, for any dual-feasible $(\bar{\lambda}, \bar{\mu}, \bar{\nu}, \bar{\eta})$, the **classical Benders optimality cut**

$$\theta \geq \sum_{i \neq j, t} (x_{ij} - 1) \bar{\lambda}_{ijt} - b \sum_j \bar{\mu}_j + \sum_j \sum_{t \in T_j} \bar{\nu}_{jt},$$

valid for *every* x — integer or fractional — and tight at the incumbent $\bar{x}$ by strong duality. This is the default cut of the implementation (`_build_benders_cut_sparsepair`); it binds on a whole half-space of sequences, unlike the no-good cut below.

The *combinatorial* alternative (`use_combinatorial_cuts`): given an integer incumbent with sequence σ and arc set $A(\sigma) = \{(i, j) : \bar{x}_{ij} = 1\}$ (the depot cycle, $|A(\sigma)| = n + 1$), evaluate $q = \text{KTNS}(\sigma)$; if $\theta < q$ add:

Proposition 3.1 (Validity of the combinatorial optimality cut). *For the sequence σ with $q = \text{KTNS}(\sigma)$ and $|A(\sigma)| = n + 1$ (the depot cycle), the inequality*

$$\theta \geq q \left(1 - \sum_{(i,j) \in A(\sigma)} (1 - x_{ij}) \right) = q \left(\sum_{(i,j) \in A(\sigma)} x_{ij} - n \right)$$

is valid: it forces $\theta \geq q$ whenever the master reproduces σ exactly, and is slack (≤ 0 on the right) for every other sequence. (This matches the implementation: `rhs = q(1 - n_arcs)`)

with $n_arcs = n + 1$.)

Proof. If x reproduces σ then $\sum_{(i,j) \in A(\sigma)} x_{ij} = n + 1$, the right-hand side is q , and $\theta \geq q = \text{KTNS}(\sigma)$ is exactly the subproblem optimum (theorem 1.1), hence valid. If x omits at least one arc of σ then $\sum_{(i,j) \in A(\sigma)} x_{ij} \leq n$, the right-hand side is $\leq 0 \leq \theta$, so the cut imposes nothing. No feasible sequence is wrongly excluded. $\square$

These “no-good” cuts are valid but weak. Each cut binds only on the single sequence that generated it, so naively BBC could enumerate exponentially many sequences. Two strengthenings are used/possible: (i) the always-on pairwise bound (4) keeps the root bound non-trivial; (ii) *cut lifting* — because $\text{KTNS}(\sigma)$ depends only on the relative order in which tools are first/last needed (its 0-block structure, cf. Part I), a single evaluation certifies the same bound for the whole family of sequences sharing that structure, yielding cuts of the form $\theta \geq q(\sum_{(i,j) \in S} x_{ij} - |S| + 1)$ for a *subset* $S \subseteq A(\sigma)$ of order-critical arcs. Identifying minimal such S is the practical lever for BBC performance.

The Algorithm

BBC is a single branch-and-bound tree on the master with a lazy-constraint callback:

Branch-and-Benders-cut for the uniform SSP (integer cuts).

1. Build the master (1)–(4); install a lazy-constraint callback.
2. At each integer incumbent (x, θ) :
 - (a) extract the sequence $\sigma \leftarrow \sigma(x)$; if x contains a subtour, add a subtour cut and reject;
 - (b) **default:** solve the DSP at $\bar{x}$ (an LP; one model per thread, objective coefficients updated per call), obtaining value q and duals $(\bar{\lambda}, \bar{\mu}, \bar{\nu}, \bar{\eta})$; **option:** $q \leftarrow \text{KTNS}(\sigma)$ ($O(n|T|)$, exact by theorem 1.1);
 - (c) if $\theta < q$, add the classical dual cut (default) or the cut of proposition 3.1 (combinatorial mode) and reject the incumbent;
 - (d) otherwise accept it — a genuine optimal-tooling schedule of cost $q = \theta$.
3. Continue until the tree is exhausted; return the incumbent, of cost Z_{SSP}^* .

Theorem 4.1 (Exactness and finiteness). *The integer-cut BBC of section 4 terminates and returns Z_{SSP}^* .*

Proof. Every accepted incumbent satisfies $\theta \geq q$, where q is the true tooling optimum of σ : in combinatorial mode $q = \text{KTNS}(\sigma)$ by theorem 1.1; in default mode q is the DSP value, which equals the tooling LP value by strong duality and hence the integer tooling optimum by the integrality of the tooling LP at Hamiltonian $\bar{x}$ (section 3; numerically confirmed). Thus the returned value is attainable and $\geq Z_{\text{SSP}}^*$. Conversely both cut families are valid for every x — the dual cut by weak duality (the dual feasible set does not depend on x), the combinatorial cut by proposition 3.1 — so the master remains a relaxation and its optimum is $\leq Z_{\text{SSP}}^*$. Finiteness: there are finitely many sequences; each rejection adds a cut that is tight at the rejected $(\bar{x}, \theta)$ and enforces $\theta \geq q$ there permanently, so no sequence is rejected

twice. □

Position Among the Exact Methods

Method	Subproblem	Subproblem cost	
Compact ILP (Part I)	—	—	
DW / column generation (Part II)	pricing (set-union knapsack / SSP)	NP-hard	
Cluster-MTZ (Part IV)	—	—	no (settled)
Branch-and-Benders (this Part)	tooling via KTNS	poly $O(n T)$	

BBC is the only route here whose subproblem is provably polynomial and whose exactness is unconditional. Its weakness: cuts are separated only at integer incumbents (until fractional separation is validated), so the node LP bound may improve slowly; the combinatorial cuts in particular bind on single sequences. This is the mirror image of Part IV (cheap polynomial constraints, but a conditional/heuristic model) and of Part II (a strong LP bound, but an NP-hard subproblem).

Open Direction: Fractional Benders Cuts

OP (fractional Benders cuts; cf. 09_open_problems.tex, OP 8). Fractional separation *exists* canonically: the DSP cut of section 3 is valid at any fractional $\bar{x}$, and the relaxation handler (`use_fractional_cuts`, `handle_relaxation`) implements it — *untested* (it requires presolve off and the thread-safe worker-LP-reuse pattern of `bendersatsp2.py`). The open questions are: (i) *empirical* — does fractional separation pay on the Catanzaro Tabela1C benchmarks (bound improvement vs. node-LP cost, cut management)? (ii) *structural* — the DSP value at fractional $\bar{x}$ may be a weak bound; characterise its strength against the SSPMF root bound $|U| - b$, and search for *stronger* polynomially separable families (e.g. from a min-cost-flow relaxation of the KTNS recursion, or lifted combinatorial cuts over order-critical arc subsets $S \subseteq A(\sigma)$); (iii) *triplet bounds* $w_{ijk} = \max(0, |T_i \cup T_j \cup T_k| - b)$ on consecutive triplets as static root strengthening. These are the practical levers for scaling BBC.

Remark 6.1 (Relation to the gap analysis). By the grouping-exactness theorem of Part V, $Z_{\text{SSP}}^* = \min_{\mathcal{G}} \text{GTSP}(\mathcal{G})$; BBC reaches the same optimum from the sequence side. A primal heuristic for the BBC incumbent is therefore the JGP+GSP schedule of Part V, and the bound $\theta \geq \max(K^* - 1, |U| - b)$ (Part V) may be added to (4) as an extra valid inequality to strengthen the root.

Implementation Notes

The CPLEX implementation is `src/BBC/branch_and_benders_cut_cplex.py` (public API in `branch_and_benders_cut.py`; shared subtour/sequence/KTNS utilities in `bbc_common.py`). Subtour detection and cut separation run in the lazy callback.

Objective convention (corrected). The DSP enforces an *empty* depot magazine ($y_{0t} = 0$), so the BBC objective counts *all* insertions, including the first job’s load — the empty-start convention, not the free-initial-load convention of Parts I and V. The two are equivalent up to a *sequence-independent* constant: by the cost identity of Part V (05, Lemma “Cost

identity”), empty-start cost = free-initial cost + $\min(b, |U|)$ for every sequence, since exactly $\min(b, |U|)$ first-time insertions are absorbed by a free initial fill. Hence the *argmin* is *unaffected* and BBC is exact in both conventions, but reported objectives must be shifted by $\min(b, |U|)$ when compared against free-initial baselines. **Verified consistency (second audit):** the cell-6b comparison and the entire benchmark pipeline (including the JGP+GSP precompute) use the empty-start convention uniformly, so the cross-solver tables and the *gap* columns are correct as-is (gaps are convention-invariant). What is *not* invariant is the *ratio*: H/Z_{SSP}^* computed from benchmark CSVs is deflated by the common shift and must be converted (subtract $\min(b, |U|)$ from both) before comparison with the free-initial theory of Part V. Two external caveats remain: the GTSP reference solver (notebook cell 6) is free-initial, so its objective differs from the cell-6b table by exactly $\min(b, |U|)$; and the *published* conventions of Laporte (2004) and da Silva (2024) should be verified before quoting their tables against repo numbers (the repo implementations force an empty depot magazine, which may deviate from the papers).

Comparison baselines (LSS, SSPMF) are in `lss_formulation.py` and `sspmf_formulation.py`.

References

- [Catanzaro et al.(2015)Catanzaro, Gouveia, and Labbé] D. Catanzaro, L. Gouveia, and M. Labbé. Improved integer linear programming formulations for the job sequencing and tool switching problem. *European Journal of Operational Research*, 244(3):766–777, 2015.
- [Crama et al.(1994)Crama, Kolen, Oerlemans, and Spieksma] Y. Crama, A. W. J. Kolen, A. G. Oerlemans, and F. C. R. Spieksma. Minimizing the number of tool switches on a flexible machine. *International Journal of Flexible Manufacturing Systems*, 6(1):33–54, 1994.
- [Laporte et al.(2004)Laporte, Salazar-González, and Semet] G. Laporte, J. J. Salazar-González, and F. Semet. Exact algorithms for the job sequencing and tool switching problem. *IIE Transactions*, 36(1):37–45, 2004.
- [Tang and Denardo(1988)] C. S. Tang and E. V. Denardo. Models arising from a flexible manufacturing machine, part i: Minimization of the number of tool switches. *Operations Research*, 36(5):767–777, 1988.